

Домашнее задание — ёмкость слоя линейного внимания

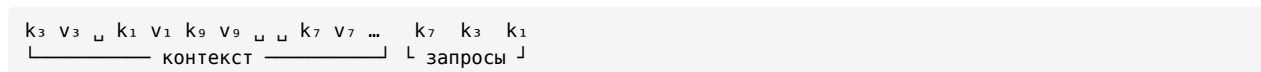
В этом задании мы измерим, сколько ассоциаций «ключ — значение» может хранить слой линейного внимания. У softmax-внимания KV-кэш растёт с длиной контекста. Вся память линейного слоя, в отличие от него, — матрица $S \in \mathbb{R}^{d_k \times d_v}$ фиксированного размера, поэтому число ассоциаций, которые она различает, ограничено. Требуется вывести это ограничение как функцию d_k и проверить его экспериментом.

- **Что сдаётся:** сначала часть 1 — вывод и гипотеза, затем отчёт на 3–4 страницы. Оценка ставится за отчёт.
- **Время:** 5–8 часов работы и вычисления без присмотра: около 40 минут на GPU, 3–4 часа на 16 потоках CPU, до 16 часов на 4 потоках. GPU не обязателен, но, как вы понимаете, желателен если не хотите застрять надолго.

1. Задача MQAR

Модель решает задачу многозапросного ассоциативного воспроизведения (multi-query associative recall, MQAR). Словарь фиксирован: 256 токенов-ключей, 256 токенов-значений и один паддинг-токен.

Последовательность состоит из контекста и запросов. Контекст — пары «токен-ключ, токен-значение», между которыми стоят паддинг-токены. Запросы — токены-ключи из этих пар. На позиции каждого запроса модель должна выдать токен-значение из его пары.



Состав последовательности задают четыре числа:

- M — число пар, о которых спрашивают; пара занимает два токена;
- D — число токенов-дистракторов: $D/2$ пар, о которых не спрашивают;
- P — число паддинг-токенов между парами;
- $Q = 8$ — число запросов, по одному токenu на запрос.

Длина последовательности равна $N = 2M + D + P + Q$. Каркас подбирает P так, что $N = 128$ при любом M , поэтому длина контекста не меняется вместе с M .

Точность — доля запросов, на которые модель выдала верное значение. Случайное угадывание даёт $1/256$, выдача любого значения из контекста — $1/(M + D/2)$, то есть 0.125 при $M = 8$.

Точка коллапса M^* — наибольшее M , при котором точность на отложенной выборке не ниже 0.50 . Используйте это определение, иначе M^* нельзя будет сравнить с результатами других отчётов.

M пробегает отрезок $[8, 32]$. Меньше $Q = 8$ оно быть не может: запросы — различные ключи из пар. Больше 32 оно быть не может: примерно при $M + D/2 > 4Q = 32$ задачу не выучивает даже softmax-внимание. Поэтому размерности головы в задании малы: $d_k \in \{8, 10, 12, 16\}$.

2. Каркас

Каркас — пакет `linattn.homework` из публичного репозитория [linear-attention-public](#). Он содержит всё, кроме кода аппроксимации, и править его не нужно.

- **Данные** (`data.py`) синтетические и генерируются заново для каждого запуска: 32 768 обучающих последовательностей и 2 048 отложенных из независимого потока случайных чисел. Обучающая выборка конечна, поэтому точность на обучающей и на отложенной выборках может различаться.
- **Модель** (`model.py`) состоит из эмбедингов токенов и позиций и двух блоков, в каждом из которых слой внимания и MLP. Ширина модели 128, одна голова, $d_v = 64$. Слой внимания задаётся ветвью:
 - ▶ softmax — softmax-внимание, контроль;
 - ▶ linear — $S_t = S_{t-1} + k_t v_t^\top$;
 - ▶ delta — $S_t = (I - \beta_t k_t k_t^\top) S_{t-1} + \beta_t k_t v_t^\top$, где $\beta_t \in (0, 1)$ вычисляется по входу слоя.

Чтение в линейных ветвях — $o_t = S_t^\top q_t$. Векторы q и k нормированы на единичную длину, v — нет. После проекций q, k, v стоит каузальная свёртка по времени ширины 4, выход слоя проходит RMSNorm.

- **Обучение** (train.py): AdamW, 2000 шагов, батч 64. Гиперпараметры заморожены.
- **Поиск M^*** (sweep.py) сначала измеряет точность в четырёх равноотстоящих точках отрезка $[8, 32]$, затем уточняет M^* бисекцией с точностью до 4.
 - Точка **цензурирована**, если M^* лежит вне отрезка. censored: "below" — ветвь не проходит порог уже при $M = 8$, censored: "above" — проходит его и при $M = 32$.
 - Флаг non_monotone означает, что точность после падения снова выросла. Это сбой оптимизации, а не предел памяти.

3. Порядок работы

Каждый студент прогоняет одно правило обновления, linear или delta, с двумя собственными seed. Правило и seed выводятся из фамилии, записанной латиницей так же, как в ведомости, — выбрать правило нельзя.

1. В первый день — установка, проверка gate и запуск перебора.
2. Пока идёт перебор — часть 1. Гипотеза записывается до просмотра результатов перебора и сдаётся вместе с выводом.
3. После срока сдачи части 1 публикуются эталонные результаты — сетка reduced для обоих правил с seed 0 и 1. Архив reference.zip появится в [пелизах linear-attention-public](https://github.com/voorhs/linear-attention-public). Он распаковывается в results/, эталон ложится в results/reference/grid_reduced.json.
4. Затем часть 2, одно расширение и отчёт.

```
pip install "linattn[plot] @ git+https://github.com/voorhs/linear-attention-public@v0.2.0"
python -m linattn.homework.cli assign --student ivanov # правило и seed
python -m linattn.homework.cli gate # контроль, 2 запуска
python -m linattn.homework.cli time-run --arm linear --n-pairs 16 --d-k 12 # секунд на запуск
python -m linattn.homework.cli budget --student ivanov --seconds-per-run 248 # подставить своё число
python -m linattn.homework.cli sweep --student ivanov # перебор
python -m linattn.homework.cli plot --grid student --out figures # рисунки
```

Вместо ivanov подставьте свою фамилию.

gate обучает softmax-внимание при $M = 8$ и $M = 32$. Проверка пройдена, если в обеих точках точность на отложенной выборке выше 0.95. У softmax-внимания нет предела памяти, поэтому провал контроля означает, что модель не выучивает саму задачу. sweep не запускается, пока gate не пройден.

Сетка студента — 42 обучающих запуска:

- ёмкость: своё правило, $d_k \in \{8, 10, 12, 16\}$, два seed, до пяти запусков на поиск M^* ;
- softmax-контроль: $d_k = 16$, $M \in \{8, 32\}$.

Время — $(42 + 2) \times$ секунд на запуск, с учётом gate:

- 248 с на 16 потоках CPU дают 3 часа;
- 1057 с на 4 потоках — 13 часов;
- 43 с на RTX 3060 Laptop — 32 минуты.

Каждый запуск кэшируется в results/runs/, повторный вызов sweep продолжает с места остановки. Итог записывается в results/grid_student.json.

В Colab команды запускаются из ячейки с префиксом !. При отключении сессии диск стирается вместе с кэшем. Поэтому каталоги результатов следует держать на Google Drive: флаги --out, --summary-dir и --gate-out у sweep и extend и флаг --out у gate.

4. Часть 1 — вывод

Требуется вывести:

$$\text{SNR} = \frac{d_k}{W-1}, \quad W = 2M + D + Q, \quad M^* = c d_k - \frac{D + Q - 1}{2}, \quad c = \frac{1}{2\theta},$$

где W — число записей в состоянии в момент чтения, а θ — порог SNR, ниже которого чтение ошибается. При $Q = 8$ свободный член равен $-(D + 7)/2$. Наклон c не зависит от D , а значит, и от длины последовательности. Свободный член от D зависит.

Оценивается точность вывода: каждая величина определена, каждое допущение названо там, где используется, каждый переход показан. В лекции часть шагов упрощена, здесь они нужны полностью.

1. **Разложение чтения.** Запишите $S_t^\top q$ после W записей при $q = k_i$ как сумму сигнала и вектора помехи n .
2. **Мощность помехи.** Мощность — матожидание квадрата нормы, $\mathbb{E}\|n\|^2$; SNR — отношение мощностей сигнала и помехи. Допущения:
 - ключи независимы и равномерно распределены на единичной сфере в $\mathbb{R}^{d_k}$;
 - значения не зависят от ключей, $\|v_j\| = 1$.
 Покажите, почему перекрёстные члены в $\|n\|^2$ в среднем равны нулю, и выведите $\mathbb{E}[(k_j^\top k_i)^2] = 1/d_k$.
3. **Число записей W .** Укажите, для какого из Q запросов оно считается, и объясните, почему паддинг в него не входит.
4. **Прямая теории.** Найдите наибольшее M , при котором выполняется $\text{SNR} \geq \theta$.
5. **Дельта-правило.** Покажите, что при $\beta_t = 1$ и $\|k_t\| = 1$ чтение $S_t^\top k_t$ возвращает ровно v_t . Предскажите, выше или ниже прямой ветви `linear` лежат точки M^* ветви `delta`.
6. **Допущения и модель.** Для каждого допущения из п. 2 скажите, выполняется ли оно в модели из §2.
7. **Гипотеза** из двух частей:
 - значение c ;
 - сдвиг M^* при добавлении D дистракторов и при добавлении P паддинг-токенов.

Для каждой части назовите измерение, которое её опровергло бы.

5. Часть 2 — эксперимент

`plot` строит два рисунка в `figures/`:

- `accuracy-vs-m.png` — точность в зависимости от M , отложенная выборка сплошной линией, обучающая пунктиром;
- `collapse-vs-dk.png` — M^* в зависимости от d_k для каждого `seed`, цензурированные точки показаны пустыми треугольниками на краю отрезка.

Самостоятельно пишется только код аппроксимации. Собственные данные — `results/grid_student.json`, эталон — `results/reference/grid_reduced.json`; в обоих список `capacity`. Каждая запись содержит `arm`, `d_k`, `seed`, `m_star`, `censored`, `non_monotone` и `evaluated`. В `evaluated` для каждого измеренного M лежат `n_pairs`, `eval_acc` и `train_acc`.

Аппроксимация должна удовлетворять условиям:

- прямая $M^* = c d_k + b$ строится методом наименьших квадратов со свободным членом — по собственным данным и по эталону для каждой ветви;
- c и b приводятся по каждому `seed` вместе с разбросом;
- цензурированные точки исключены из подгонки и перечислены в тексте;
- точки с `non_monotone` обсуждаются отдельно;
- одним предложением объяснено, почему подгонка через начало координат исказила бы c и как это выглядело бы на рисунке.

Рисунки нужно дополнить:

- на collapse-vs-dk.png — прямыми аппроксимации, прямой теории из части 1 и эталонными точками обеих ветвей;
- на accuracy-vs-m.png — точностью softmax-контроля (список softmax_control того же файла).

В тексте сравните c с предсказанием, а $b - c - (D + 7)/2$. Сравните собственные M^* с эталоном для того же правила и проверьте по эталону предсказание п. 5 части 1 о delta и linear. Затем прочитайте пары кривых. Предел памяти снижает обучающую и отложенную точность вместе: состоянию безразлично, встречалась ли последовательность при обучении. Если обучающая точность высокая, а отложенная низкая, модель запомнила обучающие последовательности весами, и такая точка не измеряет состояние.

6. Часть 3 — одно расширение

Выберите одно из четырёх расширений и запустите его командой extend:

```
python -m linattn.homework.cli extend --student ivanov --which a --dry-run # число запусков
python -m linattn.homework.cli extend --student ivanov --which a # запуски
```

Запуски, которые уже сделала сетка студента, берутся из кэша. Итог записывается в results/extension_<буква>.json, для (a) и (b) строится ещё рисунок figures/extension-<буква>.png.

- **(a) Отрицательные контроли** — 3 запуска. Своё правило, $d_k = 16$, $M = 16$, первый seed: размер чанка 16, 32 и 64 и длина $N = 64$ и 128 за счёт паддинга.
 - ▶ Чанковая форма при любом размере чанка вычисляет ту же функцию, что и рекуррентность. Поэтому отличия по размеру чанка возникают только из порядка суммирования в плавающей точке — оцените их сверху.
 - ▶ Отсутствие эффекта паддинга — предсказание: слой может научиться ничего не записывать для одного и того же инертного токена.

Сообщите оба результата и объясните, что означало бы, если бы не прошёл только один из контролей.

- **(b) Число записей** — 9 запусков, 14 при правиле delta. Всегда linear, потому что сдвиг выведен для него: $d_k = 16$, $D \in \{0, 16, 32\}$, первый seed. Проверьте, что M^* сдвигается на $-D/2$. Сдвиг 0 соответствовал бы утверждению «ёмкость не зависит от длины», сдвиг $-D$ — подсчёту записей по парам.
- **(c) Равные байты состояния** — 10 запусков. Сравните $(d_k, d_v) = (16, 32)$ и $(8, 64)$ на своём правиле: оба состояния содержат по 512 чисел. Байты растут как $d_k d_v$, а по прямой теории M^* зависит только от d_k .
- **(d) Гибрид** — 10 запусков. Второй слой — softmax-внимание, первый — своё правило, $d_k = 8$; сравнение — с той же точкой без гибрида из сетки студента. Сообщите M^* и оба числа состояния из §7 для каждого слоя.

7. Два числа состояния

Каждая архитектура модуля описывается двумя величинами: **байты состояния на слой при длине контекста N** и **прирост байтов на токен, на слой**. В fp32 (4 байта на число) при H головах:

ветвь	байты состояния на слой при длине контекста N	прирост байтов на токен, на слой
linear, delta	$4Hd_k d_v$, не зависит от N	0
softmax	$4H(d_k + d_v)N$	$4H(d_k + d_v)$, ненулевой

Для softmax-внимания $d_k + d_v$ — ключ и значение одного токена в KV-кэше; при $d_k = d_v = d$ получается $2d$ из лекции.

Приведите оба числа для собственной конфигурации при $N = 128$ и для softmax-контроля из сетки ($d_k = 16$, $d_v = 64$) при той же длине. Формулировка «байты состояния на токен» в отчёте запрещена.

Для softmax-внимания это прирост, а для линейного слоя — величина $4Hd_kd_v/N$, которая убывает с N и ничего не описывает.

8. Отчёт

Отчёт занимает 3–4 страницы. Разделы идут в таком порядке:

1. **Предсказание:** вывод части 1 и гипотеза.
2. **Дизайн:** что меняется, что фиксировано, как проверен контроль.
3. **Результат:** рисунки части 2, аппроксимация, сравнение с предсказанием.
4. **Расширение.**
5. **Два числа состояния.**
6. **Три формы** — два абзаца по лекции и семинару:
 - перенос состояния между чанками и почему внутричанковая маска включает диагональ;
 - почему рекуррентная форма, выполняющая меньше FLOP, чем параллельная, обучается медленнее; объяснение — через арифметическую интенсивность и число последовательных шагов.
7. **Что изменило бы вывод:** измерение, которое опровергло бы результат, и сделано ли оно.

У каждого рисунка есть оси, единицы и подпись, формулирующая утверждение. Числа в тексте совпадают с рисунками.

9. Оценивание

Оцениваются вывод, дизайн эксперимента и прочтение данных, а не согласие теории с экспериментом. Аккуратно выведенная гипотеза, которую эксперимент опровергает, изложенная вместе с данными и с указанием нарушенного допущения, получает полный балл.

критерий	баллы
Вывод части 1	20
Гипотеза и условия её опровержения	10
Дизайн эксперимента	15
Аппроксимация	15
Расширение	10
Два числа состояния	10
Три формы	10
Оформление отчёта	10

Баллы снимаются за:

- W , посчитанное по парам, а не по токенам;
- подгонку без свободного члена или с цензурированными точками;
- усреднение по seed без разброса;
- отложенную точность, приведённую без обучающей;
- запрещённую формулировку из §7 или два числа состояния только для одной ветви;
- утверждения, не подкреплённые рисунком или числом.